

Homework 02*Due: August 15, 2026 at 11:59 PT**August 11, 2026***Sources**

- Book: <https://dsc40b.com/materials/default/notes/book.pdf>
- Lecture 04: https://dsc-courses.github.io/DSC40B-SU26/resources/lectures/lec04/Lecture_04_blank.pdf
- Lecture 05: https://dsc-courses.github.io/DSC40B-SU26/resources/lectures/lec05/Lecture_05_blank.pdf
- Lecture 06: https://dsc-courses.github.io/DSC40B-SU26/resources/lectures/lec06/Lecture_06_blank.pdf

Instructions

Problems 1–3 are written; submit a single PDF to Gradescope. Problem 3 is a programming problem; submit `hw02.py` to the autograder. When a problem asks you to solve a recurrence by unrolling, show all four steps from lecture: unroll several levels, write the k th line, solve for k at the base case, and substitute back. Unless stated otherwise, assume n is a power of 2.

Problem 1

Recurrences.

Consider the following function.

```
1 def half_sum(arr, start=0, stop=None):
2     if stop is None:
3         stop = len(arr)
4     n = stop - start
5     if n <= 1:
6         return arr[start] if n == 1 else 0
7
8     middle = (start + stop) // 2
9     total = 0
10    for i in range(start, stop):
11        total += arr[i]
12
13    return total + half_sum(arr, start, middle)
```

Let $T(n)$ denote the worst-case number of basic operations performed by `half_sum` on an input range of size $n = \text{stop} - \text{start}$.

- (a) Write the recurrence for $T(n)$, including the base case. Justify the local-work term and the size of the recursive call separately.
- (b) Solve the recurrence by unrolling. Show all four steps.
- (c) State the worst-case time complexity in Θ notation.
- (d) Merge sort satisfies $T(n) = 2T(n/2) + n$ and solves to $\Theta(n \log n)$. The function above performs the same amount of local work and recurses on a subproblem of the same size. In two sentences, explain why the two answers differ.
- (e) Suppose the last line is changed to `return total + half_sum(arr, start, middle) + half_sum(arr, middle, stop)`. Give the new recurrence and its solution.

Problem 2

(a) Tracing

Let `arr = [8, 3, 2a, 1, 2b, 2c]` (note: they are all 2's, but the subscript helps you denote for *stableness*)

- (1) Show the contents of `arr` after each iteration of the outer loop of `selection_sort`. Mark the barrier index at each step.
- (2) Do the same for `insertion_sort`.
- (3) For `insertion_sort` on this input, how many *writes* to `arr` does each outer iteration perform? Sum them, and relate the total to the worst-case bound proved in lecture.

(b) A hybrid sort

Consider the following algorithm. Assume n is divisible by k , and that `merge_all` merges the n/k sorted blocks pairwise, bottom-up, using the `merge` routine from lecture.

```
1 def blocksort(arr, k):  
2     for i in range(0, len(arr), k):  
3         insertion_sort(arr[i:i+k])    # sort each block of size k  
4     merge_all(arr, k)                 # merge the n/k sorted blocks
```

- (1) Give the worst-case cost of the insertion-sort phase in terms of n and k .
- (2) Give the worst-case cost of the merge phase in terms of n and k . (How many rounds of merging are there, and what does each round cost?)
- (3) Give the total worst-case cost in terms of n and k .
- (4) For which values of k is the total $\Theta(n \log n)$? State what happens at the two extremes $k = n$ and $k = 1$, and identify the familiar algorithm each extreme reduces to.
- (5) The loop passes `arr[i:i+k]` to `insertion_sort`. Name the bug this introduces, and cite the fact about Python slicing from lecture that explains it. What is the additional cost of the copying, and does it change your answer to part (3)?

Problem 3

Submit all four functions in a single file named `hw02.py`. You may not use `sorted()`, `list.sort()`, `arr.count()`, `list.index()`, or the `bisect` module anywhere in these problems.

Part 1: Counting occurrences in $\Theta(\log n)$

Each function below must run in $\Theta(\log n)$ time on a sorted array of n elements; the autograder includes a timing test that a linear scan will fail.

```
1 def leftmost(arr, target):
2     """Return the index of the first occurrence of target in sorted arr,
3     or None if target does not appear.
4
5     Must run in Theta(log n) time.
6
7     >>> leftmost([1, 2, 2, 2, 5], 2)
8     1
9     >>> leftmost([1, 2, 2, 2, 5], 3) is None
10    True
11    >>> leftmost([], 4) is None
12    True
13    """
14    ...
15
16
17 def rightmost(arr, target):
18     """Return the index of the last occurrence of target in sorted arr,
19     or None if target does not appear.
20
21     Must run in Theta(log n) time.
22
23     >>> rightmost([1, 2, 2, 2, 5], 2)
24     3
25     >>> rightmost([7, 7, 7], 7)
26     2
27     """
28     ...
29
30
31 def count_occurrences(arr, target):
32     """Return the number of times target appears in sorted arr.
33
34     Must run in Theta(log n) time.
35
36     >>> count_occurrences([1, 2, 2, 2, 5], 2)
37     3
38     >>> count_occurrences([1, 2, 2, 2, 5], 4)
39     0
40     >>> count_occurrences([2] * 1000, 2)
41     1000
42     """
43     ...
```

Part 2: Base- b radix sort

Implement least-significant-digit (LSD) radix sort for an arbitrary base b , using the stable counting-sort subroutine from lecture as the per-digit pass.

Assume the list contains only *non-negative* integers. The sort must be performed in place: mutate `lst` and return `None`. Do not sort the buckets, and do not compare two keys to each other at any point.

```
1 def radix_sort(lst, base=10, descending=False):
2     """Sort a list of non-negative integers in place, using LSD radix sort
3     with the given base.
4
5     Parameters
6     -----
7     lst : list of int
8         The list to sort. Mutated in place. All entries are assumed to be
9         non-negative integers.
10    base : int
11        The base whose digits are used for bucketing. Must be at least 2.
12        Raise ValueError otherwise.
13    descending : bool
14        If True, sort from largest to smallest; otherwise smallest to
15        largest.
16
17    Returns None.
18
19    >>> arr = [170, 45, 75, 90, 2, 802, 24, 66]
20    >>> radix_sort(arr)
21    >>> arr
22    [2, 24, 45, 66, 75, 90, 170, 802]
23
24    >>> arr = [5, 3, 9, 1, 0]
25    >>> radix_sort(arr, base=2)
26    >>> arr
27    [0, 1, 3, 5, 9]
28
29    >>> arr = [5, 3, 9, 1, 0]
30    >>> radix_sort(arr, base=16, descending=True)
31    >>> arr
32    [9, 5, 3, 1, 0]
33
34    >>> arr = []
35    >>> radix_sort(arr)
36    >>> arr
37    []
38
39    >>> radix_sort([1, 2], base=1)
40    Traceback (most recent call last):
41    ...
42    ValueError: base must be at least 2
43    """
44    ...
```